Longest common substring (DP)

```
# Time c. O(nm), space c. O(nm)
def longest_common_substring(s1, s2):
    m = [[0] * (1+len(s2)) for i in\
                    range(1+len(s1))]
    max_len, end_idx = 0, 0
    for i in range(1, 1 + len(s1)):
        for j in range(1, 1 + len(s2)):
            if s1[i - 1] == s2[j - 1]:
                m[i][j] = m[i-1][j-1] + 1
                if m[i][j] > max_len:
                    max_len = m[i][j]
                    end_idx = i
            else:
                m[i][j] = 0
    return s1[end_idx-max_len : end_idx]
```

Make sentences with dictionary

```
def make_sent(string, dictionaries):
    global count
    if len(string) == 0:
        return True
    for i in range(0, len(string) + 1):
        prefix,suffix=string[:i],string[i:]
        if prefix in dictionaries:
            if suffix in dictionaries or\
               make_sent(suffix, dictionaries):
                count += 1
    return True

count = 0
string1 = "applet"
dictionary1 = {'app','let','apple','t','applet'}
print( make_sent(string1, dictionary1) )
print( count )
```

```
True
3
```

Longest common subsequence (DP)

```
# time c. O(nm), space c.
def lcs(s1, s2):
    matrix = [ ['' for x in range(len(s2))]\
               for y in range(len(s1)) ]
    for i in range(len(s1)):
        for j in range(len(s2)):
            if s1[i] == s2[j]:
                if i == 0 or j == 0:
                    matrix[i][j] = s1[i]
                else:
                    matrix[i][j] = matrix[i-1][j-1] +\
                                   s1[i]
            else:
                matrix[i][j] = max( matrix[i-1][j],
                                    matrix[i][j-1],
                                    key=len)
    return matrix[-1][-1]
```

Longest common substring in array

```
# time c. O(n*n*(n + n-1 + n-2 ... etc.))?
def longest_substr(arr):

    if len(arr) <= 1:
        return ''
    res       = ''
    reference = arr[0]
    for i in range(len(reference)):
        for j in range(len(reference)-i+1):
            candidate = reference[i:i+j]
            if j > len(res) and\
               all(candidate in x for x in arr):
                res = candidate
    return res
```

Max subarray sum (sum + indices)

```
def max_subarray_sum(arr):
    if len(arr)==0: return 0
    max_sum = curr_sum = arr[0]  # if neg
    start, tstart, end = 0, 0, 0
    for i in range(1, len(arr)):
        if arr[i] > curr_sum + arr[i]:
            curr_sum = arr[i]
            tstart = i
        else:
            curr_sum += arr[i]
        if curr_sum > max_sum:
            max_sum = curr_sum
            start = tstart
            end = i
    return max_sum, arr[ start:end+1 ]
```

All contig. subarrays w/sum k (neg or pos)

```
# Time c. O(n), space c. O(n)
from collections import defaultdict
def findSubarraysWithSumK(arr, k):
    hash_map = defaultdict(list)  # key=sum, value=end indices
    res      = []                 # subarrays
    curr_sum = 0
    for i in range( len(arr) ):
        curr_sum += arr[i]
        if curr_sum == k:
            res.append(arr[:i+1]) # or res.append((0, i+1))
        if curr_sum-k in hash_map:
            for value in hash_map[ curr_sum-k ]:
                res.append( arr[ value+1:i+1 ] )
                # or res.append((value+1, i+1))
        hash_map[ curr_sum ].append(i)
    return res
```

All non-contig. pairs sum up to k (unsorted)

```
def pair_sum(arr, k):
    if len(arr) < 2: return
    seen, output = set(), set()
    for num in arr:
        if k-num not in seen:
            seen.add(num)
        else:
            output.add( (min(num,k-num),  max(num,k-num)) )
    return '\n'.join(map(str,list(output)))
```

Non-contig pair, sum closest to k (sorted)

```
# time c. O(n)
MAX_VAL = 1000000000
def closest_sum(arr, n, k):
    res = 0, 0
    l, r, diff = 0, n-1, MAX_VAL
    while l < r:
        if abs(arr[l] + arr[r] - k) < diff:
            res = l, r
            diff = abs(arr[l] + arr[r] - k)
        if arr[l] + arr[r] > k:
            r -= 1
        else:
            l += 1
    return arr[res[0]], arr[res[1]]
```

Partition - 2 subsets w/equal sum?

```
# t.c.= O(n*sum), space=O(sum)
def isSubsetSum_dp(arr, n):
    sum_ = 0
    for i in range(n): sum_ += arr[i]
    if sum_ % 2 != 0:  return 0
    part = [0] * ((sum_ // 2) + 1)  # 0 (False)
    # part[j]=true if exists subset w/sum=j
    for i in range(n):
        for j in range(sum_ // 2, arr[i]-1, -1) :
            if (part[j - arr[i]] == 1 or j == arr[i]):
                part[j] = 1
    return part[sum_ // 2]
```

Is there subset whose sum=sum_?

```
# t.c.=2**n - each elem included or excluded
#space=O(n)-call stack in rec.=depth up to n
def isSubsetSum(arr, n, sum_):
    if sum_ == 0: # base case 1 - always possible
        return True
    if n == 0 and sum_ != 0:
        return False    # base c. 2 - no elems left
    if arr[n-1] > sum_:    # ignore last elem
        return isSubsetSum(arr, n-1, sum_)
    return isSubsetSum(arr, n-1, sum_) or\
           isSubsetSum(arr, n-1, sum_-arr[n-1])
```

Smallest +int != sum of a subset

- Sort arr. If arr[i] > res => found a gap
- Elems after arr[i] > res. res = solution

[1, 2, 5] => 4

```
# time c. O(n) - sorted array
def findSmallest(arr, n):
    res = 1
    for i in range(n):
        if arr[i] <= res:
            res = res + arr[i]
        else:
            break
    return res
```

Partition - two subsets w/min diff. sums

n * sumTotal w/memoization

```
# t.c.= O(2**n)-each step, 2 cases. Space=max depth of rec.=n
def findMinRec(arr, i, sumCalculated, sumTotal):
    # If last elem, sum of one subset = sumCalculated,
    # sum of the other=(sumTotal-sumCalculated). Take abs diff
    if i == 0:
        return abs( (sumTotal-sumCalculated) - sumCalculated )
    # For each arr[i-1], (1) we can exclude it from first set,
    # (2) we can include it. return min of two choices
    return min( findMinRec( arr,
                            i-1,
                            sumCalculated + arr[i-1],
                            sumTotal),
                findMinRec( arr,
                            i-1,
                            sumCalculated,
                            sumTotal ) )

def findmin_recursive(arr,  n):
    # total sum of arr
    sumTotal = 0
    for i in range(n):
        sumTotal += arr[i]
    return findMinRec(arr, n, 0, sumTotal)
```

res

memo[(i, sumCalc)]=res

include memo in each call

Balanced parenth. check

```
def balance_check(s):
    if len(s)%2 != 0: return False
    opening = set('([{')
    # matching pairs
    matches = set([ ('(',')'),
                    ('[',']'),
                    ('{','}'), ])
    stack = []
    for paren in s:
        if paren in opening:
            stack.append(paren)
        else:
            if len(stack) == 0:
                return False
            last_open = stack.pop()
            if (last_open,paren) not in matches:
                return False
    return len(stack) == 0
```

All non-contig. triplets w/ sum < k

```
# Time c. O(n^2); sort() + smart iteration
def count_triplets(arr, k):
    arr.sort()
    n     = len(arr)
    count = 0
    for i in range(n-2):
        start = i + 1
        end   = n - 1
        while start < end:
            if (arr[i] + arr[start] + arr[end]) >= k:
                end -= 1
            else:
                count += (end-start)    # array sorted
                start += 1
    return count
```

All non-contig. triplets w/sum=0

```
# Time c. O(n^2), space c. O(1)
def findTriplets(arr):
    arr.sort()
    n     = len(arr)
    count = 0
    for i in range(n-2):
        start = i + 1
        end   = n - 1
        while start < end:
            if arr[i] + arr[start] + arr[end] == 0:
                count += 1
                start += 1
                end   -= 1
            elif (arr[i] + arr[start] + arr[end] < 0):
                start += 1
            else:
                end -= 1
    return count
```

Count paths

```
# Time c. = O(c**n) exponential
def count_paths_rec(m, n):
    if(m == 1 or n == 1): return 1
    return count_paths_rec(m-1, n) + count_paths_rec(m, n-1)

# Time & space = O(mn)
def count_paths(m, n):
    if m < 1 or n < 1: return -1
    count = [[None for j in range(n)] for i in range(m)]
    for i in range(n): count[0][i] = 1
    for j in range(m): count[j][0] = 1
    for i in range(1, m):
        for j in range(1, n):
            count[i][j] = count[i - 1][j] + count[i][j - 1]
    return count[m - 1][n - 1]

# Time O(mn), space O(n)
def count_paths_dp(m, n):
    dp = [1 for i in range(n)]
    for i in range(1, m):
        for j in range(1, n):
            dp[j] = dp[j] + dp[j - 1]
    return dp[n - 1]
```

Get all permutations of string

```
def permute(s):
    out = []
    if len(s) == 1:
        out = [s]
    else:
        for i, char in enumerate(s):
            for perm in permute(s[:i] + s[i+1:]):
                out += [char + perm]
    return out

permute('abc')
```

```
['abc', 'acb', 'bac', 'bca', 'cab', 'cba']
```

Longest Substr. w/K Chars

```python
# Time c. O(n), space c. O(1)
# At most k distinct characters
def LongestSubstring(s, k):
    n = len(s)
    if n*k==0:
        return 0
    l, r = 0, 0
    mapp = dict()
    max_len = 0
    for c in s:
        mapp[ c ] = r
        r += 1
        if len(mapp) == k+1:
            del_idx = min(mapp.values())
            del mapp[ s[del_idx] ]
            l = del_idx + 1
        max_len = max(max_len, r - l)
    return max_len
```

Edit Levenshtein Distance

```python
def levenshtein(s1, s2):
    if len(s1) > len(s2):
        s1, s2 = s2, s1
    distances = range(len(s1) + 1)
    for idx2, char2 in enumerate(s2):
        distances_ = [ idx2+1 ]
        for idx1, char1 in enumerate(s1):
            if char1 == char2:
                distances_.append( distances[ idx1] )
            else:
                distances_.append( 1 + min((distances[idx1],\
                        distances[idx1+1], distances_[-1])))
        distances = distances_
    return distances[-1]
```

Multiply strings

```python
# GET NUMBER FROM CHAR: ord(char) - ord('0')
def multiply_strings(num1, num2):
    res = 0
    for i, c1 in enumerate(num1[::-1]):
        for j, c2 in enumerate(num2[::-1]):
            res += (ord(c1)-ord('0')) *\
                   (ord(c2)-ord('0')) *\
                   (10**(i+j))
    return str(res)
num1, num2 = '123', '456'
multiply_strings(num1, num2)
```

Longest Increasing Path in Matrix

```python
# time and space = O(mn)
def longestIncreasingPath(matrix):
    def dfs(i, j):
        if not dp[i][j]:
            val = matrix[i][j]
            dp[i][j] = 1 + max(
                dfs(i - 1, j) if i and val > matrix[i - 1][j] else 0,
                dfs(i + 1, j) if i < M - 1 and val > matrix[i + 1][j] else 0,
                dfs(i, j - 1) if j and val > matrix[i][j - 1] else 0,
                dfs(i, j + 1) if j < N - 1 and val > matrix[i][j + 1] else 0)
        return dp[i][j]
    if not matrix or not matrix[0]: return 0
    M, N = len(matrix), len(matrix[0])
    dp = [[0] * N for i in range(M)]
    return max(dfs(x, y) for x in range(M) for y in range(N))
```

Longest Substr. w/Unique Ch

```python
def lengthOfLongestSubstring2(s):
    mapp = {c:0 for c in s}
    left = 0
    max_len = 0
    for right, c in enumerate(s):
        mapp[c] += 1
        while mapp[c] > 1:
            mapp[ s[left] ] -= 1
            left += 1
        max_len = max(max_len,
                      right-left+1)
    return max_len
```

Complexity classes

- O(1) constant-time algo - **no dependency on input size**. E.g. **formula** calculating an answer OR **list[idx]**.
- O(logn) algo **halves / reduces input size at each step**. Logarithmic because (log2 n) = # times to divide n by 2 to get 1.
- O(sqrt(n)) *slower than O(logn), but faster than O(n)*; sqrt(n) = sqrt(n) / n, so sqrt(n) lies **in the middle of input**.
- O(n) **iteration over the input** - accessing each input element at least once before reporting the answer.
- O(nlogn) often indicates that algo **sorts the input** OR algo uses **data structure** where *each operation takes O(logn) time*.
- O(n^2) two **nested** loops
- O(n^3) three **nested** loops.
- O(2^n) algo iterates through **all subsets of input elements**. E.g. subsets of {1,2,3}: *{1}, {2}, {3}, {1,2}, {1,3}, {2,3} and {1,2,3}*.
- O(n!) algo iterates through **all permutations of input elements**. E.g. permutations of {1,2,3}: *(1,2,3), (1,3,2), (2,1,3), (2,3,1), (3,1,2) and (3,2,1)*.

Polynomial algo - time complexity of **O(n^k)** where k is const; if k small - algo efficient. All above except O(2^n) and O(n!) are polynomial. There are many important problems with **no known polynomial (=efficient) algo**, e.g. **NP-hard** problems

133. Clone Connected Undirected Graph ¶

```python
# t.c. O(n+m),n=nodes,m=edges s.c. O(n)
# Approach: BFS where visited is dict[curr_node] = cloned_node
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors if neighbors is not None else []
def cloneGraph(start: Node) -> Node:
    if not start:
        return start
    visited, queue = {}, [start] # Dict[visited node] = clone
    visited[start] = Node(start.val, []) # Clone start
    while queue:
        vertex = queue.pop(0)
        for neighbor in vertex.neighbors: # Iterate neighbors
            if neighbor not in visited:
                visited[neighbor] = Node(neighbor.val, [])#Clone neighb
                queue.append(neighbor)
            # Append cloned neighbor
            visited[vertex].neighbors.append(visited[neighbor])
    return visited[start]
```

Merge Sort (mid)

Usage: sorting **linked lists**, inversion count in nearly sorted arr (i < j, but A[i] > A[j]), external sort (data too big for memory)

```
# time c. O(nLogn), space c. O(n)
def merge_sort(arr):
    # arr of length 1 - returned as is
    if len(arr) > 1:
        mid = len(arr) // 2
        left =  merge_sort(arr[ :mid ])
        right = merge_sort(arr[ mid: ])
        i, j, k = 0, 0, 0
        while i < len(left) and j < len(right):
            if left[i] < right[j]:
                arr[k] = left[i]
                i += 1
            else:
                arr[k] = right[j]
                j += 1
            k += 1
        while i < len(left):
            arr[k] = left[i]
            i += 1
            k += 1
        while j < len(right):
            arr[k] = right[j]
            j += 1
            k += 1
    return arr
```

Binary Search

```
# time O(Logn), space (O1)
def binary_search(arr, value):
    if len(arr) == 0: return None
    min_idx, max_idx= 0, len(arr)
    while min_idx < max_idx:
        mid = (min_idx + max_idx) // 2
        if arr[mid] == value:
            return mid
        elif arr[mid] < value:
            min_idx = mid + 1
        else: max_idx = mid
    return None
```

Quick Sort (pivot)

Pick **pivot** element (*first, last, random, median*), and **partition** array: all smaller elems before pivot, greater elements after pivot Usage: sorting arrays

```
# O(nLogn), worst case O(n^2)
# time c. O(logn), qualifies as in-place
def quick_sort(arr, low, high):
    if low < high:
        pi = partition(arr, low, high)
        quick_sort(arr, low, pi-1)
        quick_sort(arr, pi+1, high)

def partition(arr, low, high):
    pivot = arr[high]
    i = low - 1
    for j in range(low , high):
        if  arr[j] < pivot:
            i += 1
            arr[i], arr[j] = arr[j], arr[i]
    # place pivot in the middle
    arr[i+1], arr[high] = arr[high], arr[i+1]
    return i+1
```

Bin. tree

Heap Sort

Usage: sort nearly sorted array,
k largest (smallest) elems
Complete binary tree = every level filled,
except possibly last, and nodes are in far left
Binary Heap = cbt where parent >(<) children
Procedure: build max heap; replace max w/ last elem; reduce heap by 1; heapify root; repeat Array repr:
root = arr[0];
for any i-th node arr[i]:
a) arr[(i-1)/2] = parent node
b) arr[(2*i)+1] = left child
c) arr[(2*i)+2] = right child

```
# time O(nLogn), space in-place
def heap_sort(arr):
    n = len(arr)
    for i in range(n, -1, -1): # build heap
        heapify(arr, n, i)
    for i in range(n-1, 0, -1):
        arr[i], arr[0] = arr[0], arr[i]
        heapify(arr, i, 0)   # extract elems

def heapify(arr, n, i):
    largest = i             # root
    l = 2 * i + 1
    r = 2 * i + 2
    if (l < n and arr[i] < arr[l]):
        largest = l         # if left child exists
    if (r < n and arr[largest] < arr[r]):
        largest = r         # if right exists
    if (largest != i):
        arr[i],arr[largest] = arr[largest
        heapify(arr, n, largest)
```

Queue with two stacks

```
class Queue2Stacks(object):
    def __init__(self):
        self.instack  = []
        self.outstack = []

    def enqueue(self,element):
        self.instack.append(element)

    def dequeue(self):
        if not self.outstack:
            while self.instack:
                self.outstack.append(
                    self.instack.pop()
                )
        return self.outstack.pop()
```

Next greater elem w/stack

[11, 14, 21] => 11:14, 14:21, 21:-1

```
# time O(n)
def printNGE(arr):
    stack = []
    stack.append(arr[0])
    for i in range(1, len(arr)):
        next_ = arr[i]
        if stack:
            elem = stack.pop()
            while elem < next_ :
                print(str(elem)+ " :  " + str(next_))
                if not stack:
                    break
                elem = stack.pop()
            if  elem > next_:
                stack.append(elem)
        stack.append(next_)
    while stack:
            elem = stack.pop()
            next_ = -1
            print(str(elem) + " : " + str(next_))

arr = [11, 14, 21, 3]
printNGE(arr)
```

Queue - FIFO, stack - LIFO
Dequeue – double-ended queue

Stack using two queues

```
from queue import Queue
class Stack:
    def __init__(self):
        self.q1 = []
        self.q2 = []
        self.curr_size = 0

    def push(self, x):
        self.curr_size += 1
        self.q2.append(x)
        while self.q1:
            self.q2.append(self.q1.pop(0))
        self.q = self.q1
        self.q1 = self.q2
        self.q2 = self.q

    def pop(self):
        if not self.q1:
            return
        self.curr_size -= 1
        return self.q1.pop(0)

    def peek(self):
        if not self.q1:
            return -1
        return self.q1[0]

    def size(self):
        return self.curr_size
```

Graphs

DFS

queue.pop(0) for BFS

```
# time c. O(V+E), space c. O(V)
def dfs(graph, start):
    visited, stack = set(), [start]
    while stack:
        vertex = stack.pop()
        if vertex not in visited:
            print(vertex, end=' ')
            visited.add(vertex)
            stack.extend(graph[vertex] - visited)
    return visited

graph = { 0: set([1, 2]), 1: set([2]),
          2: set([3]),    3: set([1, 2]) }
dfs(graph, 0)
```

```
0 2 3 1
```

BFS2

Textbook BFS + more efficient – enqueues node once. Using pop() doesn't make it DFS! (becomes just some graph traversal)

```
def bfs2(graph, start):
    visited, queue = {start}, [start]
    while queue:
        vertex = queue.pop(0)
        print(str(vertex) + " ", end="")
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

graph = {0: [1, 2], 1: [2], 2: [3], 3: [1, 2]}
bfs2(graph, 0)
```

```
0 1 2 3
```

All Paths from Vertex A to Vertex B

```
def bfs_paths(graph, start, goal):
    queue = [(start, [start])]
    while queue:
        (vertex, path) = queue.pop(0)
        for nxt in graph[vertex] - set(path):
            if nxt == goal:
                yield path + [nxt]
            else:
                queue.append((nxt, path + [nxt]))
```

```
def toposort(graph):
    topo = []
    in_degree = {node: 0 for node in graph}
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1
    q = [node for node, deg in in_degree.items() if deg == 0]
    while q:
        node = q.pop()
        topo.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                q.append(neighbor)
    return topo if len(topo) == len(graph) else []
```

Articulation point (vertex) or **bridge** (edge) – disconnects graph when removed. Connectedness verified by DFS

Shortest path

Always returned first by BFS

```
def shortest_path(graph, start, goal):
    try:
        return next(bfs_paths(graph, start, goal))
    except StopIteration:
        return None

graph = {'A': set(['B', 'C']),
         'B': set(['A', 'D', 'E']),
         'C': set(['A', 'F']),
         'D': set(['B']),
         'E': set(['B', 'F']),
         'F': set(['C', 'E'])}

shortest_path(graph, 'A', 'F')
```

```
['A', 'C', 'F']
```

Graphs

- **vertices / nodes** w/names (keys) and payloads (additional info);
- unordered = **undirected graph** vs.**directed graph** (digraph);
- **weight** on edge - cost from 1 vertex to the other (distance betw. cities)
- **Path** - # vertices connected by edges.
- **Path length** - # edges in unweighted g. (n–1) OR **sum of weights** in weighted g.
- **Cycle** in directed g. - path starts / ends at same vertex.
- If no cycles - **acyclic graph**; if also directed - directed acyclic graph (**DAG**)

Adjacency matrix = 2D matrix size n×n if graph has n vertices. Each cell [i, j] indicates whether there is an edge between vertex i and vertex j (if edge - value is 1 or the weight, and 0 if no edge) - [[0, 1, 1], [1, 0, 0,], [0, 1, 0]

- **Quick edge lookup t.c.O(1)** to see if edge present, add/remove edge;
- **Simplicity of matrix**, good for matrix operations (**insights**);
- **Slow iteration - O(n)** to get vertex's _adj. list, **O(V^2)** *add vertex*
- memory hog: **space c. O(n^2)** => good for **dense graphs**
- **adjacency lists are better for most tasks**
- better for **weighted graphs**;
- **Undirected** graphs - adj. matrix **symmetric**

Adjacency list r.: edge_list = [[0,1], [0,2],] or dict[list] (node: verts) or dict[dicts] (if weights)

- **Efficient neighb. iteration** and **time c. O(1)** *get vertex's adj. list* (array indexing)
- **Efficient storage** (**space c. = O(2E)**)
- **Slow edge lookup O(E)** (E=# v-s) *if edge (i,j) exists*

Objects/pointer r.

- **time c. O(n)** *access a node*;
- **space c. O(n)** if nodes only, **O(n^2)** w/pointers;
- better for **directed graphs** (pointers);
- **high search time c.**

Incidence matrix
rows = vertices, columns = edges, entries = is vertex incident to edge (bool)

200. Number of Islands

2D grid of '1's (land) and '0's (water). Return # islands.
Adjacent '1' land plots connected horiz. or vertically.
Time c.=O(m*n), **Space c.**=O(m*n) due to recursion stack.
Each recursive call contains:

- func's local vars (indices i and j).
- pointer to the grid.
- return address, etc.

```
def numIslands(grid):
    if not grid:
        return 0
    count = 0
    for i in range(len(grid)):
        for j in range(len(grid[0])):
            # island found
            if grid[i][j] == '1':
                count += 1
                # delete found island
                dfs(grid, i, j)
    return count

def dfs(grid, i, j):
    ''' Delete found island '''
    if i<0 or j<0 or i>=len(grid) or\
    j>=len(grid[0]) or grid[i][j] != '1':
        return
    grid[i][j] = '#' #anything, but '1'
    dfs(grid, i+1, j)
    dfs(grid, i-1, j)
    dfs(grid, i, j+1)
    dfs(grid, i, j-1)
```

```
grid = [ ["1","1","0","0","0"],
         ["1","1","0","0","0"],
         ["0","0","1","0","0"],
         ["0","0","0","1","1"], ]
print(numIslands(grid))
```

```
3
```

269. Alien Dictionary (FB!)

- Alien language uses Eng ABC w/unknown order of letters
- You have list of strings - words sorted lexicographically.
- Return string of unique letters sorted lexicographically increasing
- No solution - return "". Multiple solutions - return any.
- 'abc' < 'abd', 'abc' < 'abcd'
- **Solution**: a) BFS + get letter dependencies, b) toposort
- **Time c. O(C)** where C - total length of all words in input list
- **Space c. O(1)** or O(U + \min(U^2, N))O(U+min(U

```
def alienOrder(words):
    # BFS + letter dependencies as adj_list
    adj_list  = defaultdict(set)   # adj_list for each letter
    # in_degree of each unique letter
    in_degree = Counter({c:0 for word in words for c in word})
    for first_word, second_word in zip(words, words[1:]):
        for c, d in zip(first_word, second_word):
            if c != d:    # c comes before d
                if d not in adj_list[c]:
                    adj_list[c].add(d)
                    in_degree[d] += 1
                break
        else:
            # agaist rules - shorter word should be first
            if len(second_word) < len(first_word): return ''
    # TOPOLOGICAL SORT
    output = []
    queue = deque([c for c in in_degree if in_degree[c] == 0])
    while queue:
        c = queue.popleft()
        output.append(c)
        for d in adj_list[c]:
            in_degree[d] -= 1
            if in_degree[d] == 0:
                queue.append(d)
    # not all letters in output - cycle => no valid ordering
    if len(output) < len(in_degree):
        return ''
    return ''.join(output)
```

Trees

2a. Depth First Traversals (Binary Tree)

```
# time O(n),space O(h),h=height
class Node:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

# Root Left Right
def Preorder(root):
    if root:
        print(root.val, end=' ')
        Preorder(root.left)
        Preorder(root.right)

# Left Right Root
def Postorder(root):
    if root:
        Postorder(root.left)
        Postorder(root.right)
        print(root.val, end=' '),

# Left Root Right
def Inorder(root):
    if root:
        Inorder(root.left)
        print(root.val, end=' '),
        Inorder(root.right)
```

```
root            = Node(1)
root.left       = Node(2)
root.right      = Node(3)
root.left.left  = Node(4)
root.left.right = Node(5)
print("Preorder traversal:")
Preorder(root)
```

BFS (recursive) incl. height

```
# Recursive - time O(n^2), space O(n)
def LevelOrder_rec(root):
    h = height(root)
    for i in range(1, h+1):
        printGivenLevel(root, i)

# print nodes at a given level
def printGivenLevel(root , level):
    if root is None:
        return
    if level == 1:
        print(root.val, end=" ")
    elif level > 1 :
        printGivenLevel(root.left , level-1)
        printGivenLevel(root.right , level-1)

# compute height - number of nodes from root
def height(node):
    if node is None:
        return 0
    else :
        # Compute the height of each subtree
        lheight = height(node.left)
        rheight = height(node.right)

        #Use the larger one
        if lheight > rheight :
            return lheight+1
        else:
            return rheight+1
```

```
root = Node(1)
root.left = Node(2)
root.right = Node(3)
root.left.left = Node(4)
root.left.right = Node(5)
LevelOrder_rec(root)
```

Min-Heap Using `heapq`

How min-heap works with `heapq`:
min heap = flat array = complete binary tree where root=heap[0] (smallest value), for each a[i]: **left child=a[2i+1]** & **right child a[2i+2]** & a[i] < both; **parent=a[(i-1)//2]**
Get the min element: `heapq.heappop(heap)`.
Add elem to heap: `heapq.heappush(heap, (elem))`
Notes: `heapq` - always min-heap where heap[0] - smallest element.
Python has no max-heap: workaround - `heapq.heappush(heap, -x)`.

```
# t. O(n logn). No heap - O(n^2). S.c. O(n)
# Since no heapify() on unordered array - pop() & push are
# O(logn) & heap[0] is O(1). Task: Given set of tennis court
# bookings (start: int, end time: int) where times are int
# t0, t1, t2 & courts are 1, 2, 3 etc., return booking plan:
# {court: list of bookings} to have min number of courts.
import heapq

def booking_plan_optimal(bookings):
    """
    bookings: list of (start, end) tuples
    Returns: dict {court_number: [(start, end), ...]}
    """
    ordered = sorted(bookings)
    plan = {}        # {court_number: list of bookings}
    heap = []        # (finish_time_of_last_booking, court)
    for s, e in ordered:
        if heap and heap[0][0] <= s:
            # Earliest-freeing court available
            _, court = heapq.heappop(heap)
        else:
            # All courts busy -> open a new court
            court = len(plan) + 1
            plan[court] = []
        plan[court].append((s, e))
        heapq.heappush(heap, (e, court))
    return plan
```

Binary tree as array:
- node i
- parent: **(i-1)//2**
- left child: **2*i + 1**
- right child: **2*i + 2**

TYPES OF TREES:

- **m-ary tree** - rooted tree, each node has no more than m children
- **Binary tree** (BT) - each node has *at most two children*
- **Complete** BT = bin tree, **every level is full**, except possibly last (far **left**).
- **BST** - parent's value greater that values in left subtree & less values in right subtree
- **Binary Heap** (max / min) = cbt, parent >(<) children
- **Full** (proper or plane) BT - every node has *0 or 2 children*
- **Perfect** BT - all interior nodes have two children and all leaves have same depth
- **Infinite complete** BT - every node has two children (infinite levels)
- **Balanced** BT - L and R subtrees of each node differ in height by no more than 1
- **Degenerate (pathological)** tree - each parent has only one child = **linked list**
- **Representation**: list, dict, object

Check if BT is balanced

```
# time O(n), space c. O(h)-rec.stack: O(logn) or O(n)
def is_balanced(root):
    return -1 != get_depth(root)
# return 0 if unbalanc. else depth+1
def get_depth(root):
    if not root:
        return 0
    left  = get_depth(root.left)
    right = get_depth(root.right)
    if abs(left-right) > 1 or\
       left == -1 or right == -1:
        return -1
    return 1 + max(left, right)
```

BFS (iterative)

```
# time O(n), space O(w)
# w - max width of level (O(n) worst
# because cbt has n/2 nodes in last level)
from collections import deque
def LevelOrder(root):
    if root is None:    # base case
        return
    q = deque([root])
    while q:
        # can't use list.pop(0) as it's O(n)
        # and total time c. will be O(n^2)
        node = q.popleft()
        print(node.val, end=' ')
        if node.left is not None:
            q.append(node.left)
        if node.right is not None:
            q.append(node.right)
```

Lowest common ancestor (any BT)

```
def lca(root, p, q):
    if not root or root is p or root is q:
        return root
    left = lca(root.left, p, q)
    right = lca(root.right, p, q)
    if left and right:
        return root
    return left if left else right
```

*Time c. **O(n)**, space c. **O(h)** - recurs. stack:*
__O(logn)__ balanced tree or __O(n)__ skewed tree

Space c. O(min(h1, h2))
** **Best (balanced):** O(log min(N,M))*
** **Worst (skewed):** O(min(N,M))*

Check if 2 BTs equal

```
# time O(min(N,M)), N,M=# nodes
# space O(min(h1, h2))
def is_same_tree(p, q):
    if not p and not q:
        return True
    if p and q and p.val == q.val:
        return is_same_tree(p.left, q.left) and\
               is_same_tree(p.right, q.right)
    return False
```

Min / Max height

Time c. O(n), space c. O(h)-rec.stack: O(logn) or O(n)

```
def max_height(root):
    if not root: return 0
    return max(max_height(root.left),\
               max_height(root.right)) + 1

def min_height(root):
    if not root: return 0
    if not root.left or not root.right:
        return max(min_height(root.left),\
                   min_height(root.right))+1
    return min(min_height(root.left),\
               min_height(root.right)) + 1
```

Check if tree is BST

Time c. O(nlogn) can be O(n), space c. O(n)-store tree_vals

```
# If BST=>inorder traversal = sorted list
def inorder( root ):
    if tree != None:
        inorder(root.left)
        tree_vals.append(root.val)
        inorder(root.right)

def sort_check( tree_vals ):
    return tree_vals == sorted(tree_vals)

tree_vals = []
inorder(tree)
sort_check(tree_vals)
```

Invert a binary tree

Time c. O(n), space c. O(h)-rec.stack: best O(logn) or worst O(n)

```
def reverse(root):
    if not root: return
    root.left,root.right=root.right,root.left
    if root.left:
        reverse(root.left)
    if root.right:
        reverse(root.right)
```

Trim BST

Time c. O(n), space c. O(h)-rec.stack: best O(logn) or worst O(n)

```
def trimBST(tree, minVal, maxVal):
    if not tree: return
    tree.left=trimBST(tree.left, minVal, maxVal)
    tree.right=trimBST(tree.right, minVal, maxVal)
    if minVal<=tree.val<=maxVal:
        return tree
    if tree.val < minVal:
        return tree.right
    if tree.val > maxVal:
        return tree.left
```

BST node w/value closest to k

Time c. O(h)– 1 recurs. call per level, space c. O(h)-rec.stack

```
def closest_value(root, k):
    a = root.val
    child = root.left if k < a else root.right
    if not child:
        return a
    b = closest_value(child, k)
    return min(k-a, k-b)
```

collections.deque ("deck") - *double-ended queue* from Python's standard library. Implemented as doubly linked list of fixed-size blocks - adding or removing items is O(1) at either end. Better than list.pop(0) which is O(n).

Method	What it does	Time
`append(x)`	Add x to the right end	O(1)
`appendleft(x)`	Add x to the left end	O(1)
`pop()`	Remove and return the rightmost item	O(1)
`popleft()`	Remove and return the leftmost item	O(1)
`extend(iterable)`	Append all items to the right end	O(k)
`extendleft(iterable)`	Append items to the left end (order ends up reversed)	O(k)
`rotate(n)`	Rotate n steps right (negative n rotates left)	O(k)
`clear()`	Remove all items	O(n)
`count(x)`	Count occurrences of x	O(n)
`remove(x)`	Remove the first occurrence of x	O(n)
`reverse()`	Reverse in place	O(n)

BST nodes in given range

Time c. O(h+k) k=#nodes in range low-high,
space c. O(h)-rec.stack: best O(logn) or worst O(n)

```
def getCount(root, low, high):
    if root == None: return 0
    if root.data==high and root.data==low:
        return 1
    if root.data <= high and root.data >= low:
        return (1 + getCount(root.left, low, high) +\
                getCount(root.right, low, high))
    elif root.data < low:
        return getCount(root.right, low, high)
    else:
        return getCount(root.left, low, high)
```

257. BT: All Root-to-Leaf Paths

Leaf = node with no children. **Space c.** O(n)
T.c. *O(n) traversing + str concat* which in worst case - skewed tree = each path has length n so total t.c. can be **O(n^2)**

```
# recursive
def binaryTreePaths(root):
    def get_path(root, path):
        if root:
            path += str(root.val)
            if not root.left and not root.right:
                paths.append(path) # reached leaf
            else:
                path += '->'
                get_path(root.left, path)
                get_path(root.right, path)
    paths = []
    get_path(root, '')
    return paths

# iterative
def binaryTreePaths(root):
    if not root: return []
    paths = []
    stack = [(root, str(root.val))]
    while stack:
        node, path = stack.pop()
        if not node.left and not node.right:
            paths.append(path)
        if node.left:
            stack.append((node.left, path + '->' +\
                          str(node.left.val)))
        if node.right:
            stack.append((node.right, path + '->' +\
                          str(node.right.val)))
    return paths
```

938. Range Sum of BST

```
# t.c.O(n), s.c.O(n) best O(logn) if balanced
def range_sum(root,low,high):
    if root is None: return 0
    res = 0
    if root.val < low:
        res += range_sum(root.right,
                         low, high)
    elif root.val > high:
        res += range_sum(root.left,
                         low, high)
    else:
        res += root.val +\
               range_sum(root.left,low,high) +\
               range_sum(root.right,low,high)
    return res
```

199. BT Right Side View (Top2Bottom)

```
# t.c.O(n), s.c.O(h)=rec.stack
# average logn, worst n
def rightSideView(root):
    if root is None:
        return []
    def helper(node, level):
        if level == len(rightside):
            rightside.append(node.val)
        for child in [node.right,
                      node.left]:
            if child:
                helper(child, level+1)
    rightside = []
    helper(root, 0)
    return rightside
```

Rotate matrix clockwise, in place

```
# time O(2M) -> O(M), space O(1)
class Solution(object):  where M=# elem. in matrix = n^2
    def rotate(self, matrix):
        self.transpose(matrix)
        self.reflect(matrix)

    def transpose(self, matrix):
        n = len(matrix)
        for i in range(n):
            for j in range(i+1, n):
                matrix[i][j], matrix[j][i] =\
                matrix[j][i], matrix[i][j]

    def reflect(self, matrix):
        n = len(matrix)
        for i in range(n):
            for j in range(n//2):
                matrix[i][j], matrix[i][-j-1] =\
                matrix[i][-j-1], matrix[i][j]
```

Insert Number into BST

```
# t.c.=s.c.=O(n) worst, O(logn) avg
def insert(root, data):
    # Base case found where to insert
    if root is None:
        return TreeNode(data)
    if data < root.data:
        root.left = insert(root.left, data)
    else:
        root.right = insert(root.right, data)
    # Return (unchanged) root pointer
    return root
```

Maximum path sum

```
#Time O(n), space O(h): balanced O(logn), skewed O(n)
def max_path_sum(root):
    max_sum = float('-inf')
    def gain(root):
        nonlocal max_sum
        if not root: return 0
        # only pos values
        left  = max(gain(root.left), 0)
        right = max(gain(root.right),0)
        # update global max_sum
        new_path = root.val+left+right
        max_sum = max(max_sum, new_path)
        # return max gain if continue same path up
        return root.val + max(left, right)
    gain(root)
    return max_sum
```

114. Flatten Binary Tree to Linked List

```
# recursive, t.c. O(n), s.c. O(n)
def flattenTree(node):
    if not node:
        return None
    if not node.left and not node.right:
        return node # if leaf node, return node
    left  = flattenTree(node.left)
    right = flattenTree(node.right)
    if left: # If left subtree, modify connect-s
        left.right = node.right
        node.right = node.left
        node.left = None
    # once re-wired, return "rightmost" node
    return right if right else left

# modifies in-place
def flatten(self, root: TreeNode) -> None:
    flattenTree(root)
```

114. Flatten Binary Tree to Linked List

```
# Iterative in-place, t.c. O(n), s.c. O(1)
def flatten(self, root: TreeNode) -> None:
    if not root:
        return None
    node = root
    while node:
        if node.left:
            rightmost = node.left # find rightmost node
            while rightmost.right:
                rightmost = rightmost.right
            rightmost.right = node.right #rewire connect-s
            node.right = node.left
            node.left = None
        node = node.right # move on to right side of tree
```

426. Convert BST to Sorted Circular Doubly LL

(FB Favorite. Toposort, graph traversal r favs too)

In circular doubly linked list - predecessor of first elem is last element; successor of last elem is first elem.

Inorder trav.: L->node->R, linking all nodes into DLL

```
# t.c.=O(n),s.c.=O(h) (O(n) worst if tree skewed,
# but O(logn) on avg
def treeToDoublyList(root):
    def helper(node):
        nonlocal last, first
        if node:
            helper(node.left) #left
            if last:          # node
                # link prev node (last) w/node
                last.right, node.left= node, last
            else:
                # memorize 1st smallest node
                first = node
            last = node
            helper(node.right) #right
    if not root:
        return None
    # smallest (1st) & largest (last) nodes
    first, last = None, None
    helper(root)
    last.right,first.left = first,last # close DLL
    return first
```

Build Bin Tree from Array

Use level-order traversal to convert back to array

```python
# time = space = O(n)
class Node:
    def __init__(self, val):
        self.val = val
        self.left = self.right = None

def insertLevelOrder(arr, i, n):
    root = None
    if i < n:                       # base case
        root = Node(arr[i])
        root.left  = insertLevelOrder(arr, 2*i+1, n)
        root.right = insertLevelOrder(arr, 2*i+2, n)
    return root

arr = [1,2,3,4,5,6,None,None,None,7,8 ]
n = len(arr)
root = insertLevelOrder(arr, 0, n)
```

Delete Nodes, Return Forest

```python
def delNodes( root: Node,
              to_delete: List[int],
            ) -> List[Node]:
    res, to_delete = [], set(to_delete)
    def helper(root):
        if root:
            # next line exec. after recursion on way up
            root.left,root.right = (helper(root.left),
                                    helper(root.right))
            if root.val not in to_delete:
                return root
            res.append(root.left)  # if root is deleted
            res.append(root.right) # if root is deleted
    res.append(helper(root))
    return([ i for i in res if i ])
```

Nodes in Compl. Bin. Tree < O(n)

```python
# time=O(d^2)=O((logN)^2), d=depth, space = O(1)
def countNodes(root: TreeNode) -> int:
    if not root: return 0
    d = compute_depth(root)
    if d == 0: return 1
    # Last level nodes - 0 to 2**d-1 (left->right)
    # Bin search to check how many nodes exist
    left, right = 1, 2**d - 1
    while left <= right:
        pivot = left + (right - left) // 2
        if exists(pivot, d, root):
            left = pivot + 1
        else:
            right = pivot - 1
    # there are only left nodes on last level
    return (2**d - 1) + left

def compute_depth(node: Node) -> int:
    d = 0
    while node.left:
        node = node.left
        d += 1
    return d

def exists(idx: int, d: int, node: Node) -> bool:
    '''Return True if last level node idx exists'''
    left, right = 0, 2**d - 1
    for _ in range(d):
        pivot = left + (right - left) // 2
        if idx <= pivot:
            node = node.left
            right = pivot
        else:
            node = node.right
            left = pivot + 1
    return node is not None
```

Linked Lists

Singly Linked List (SLL)

```
class Node:
    def __init__(self, value):
        self.value = value
        self.next = None
```

Doubly Linked List (DLL)

```
class DLL_Node(object):
    def __init__(self,value):
        self.value = value
        self.next = None
        self.prev = None
```

nth to the last element

```
def nth_to_last_node(head, n):
    left_pointer  = head
    right_pointer = head
    for i in range(n-1):
        if not right_pointer.next:
            print('n > length list')
            return
        right_pointer = right_pointer.next
    while right_pointer.next:
        left_pointer  = left_pointer.next
        right_pointer = right_pointer.next
    return left_pointer.value
```

Check if cycle

```
def cycle_check(node):
    marker1 = node
    marker2 = node
    while marker2 and marker2.next:
        marker1 = marker1.next
        marker2 = marker2.next.next
        if marker2 == marker1:
            return True
    return False
```

Compare 2 strings as linked lists

```
# 0=same, 1=1st str lexicographically >,
# -1 - 2nd
class Node:
    def __init__(self, key):
        self.c = key ;
        self.next = None

def compare(list1, list2):
    while (list1 and list2 and\
           list1.c == list2.c):
        list1 = list1.next
        list2 = list2.next
    if (list1 and list2):
        return 1 if list1.c > list2.c else -1
    if (list1 and not list2):
        return 1
    if (list2 and not list1):
        return -1
    return 0

list1 = Node('g')
list1.next = Node('o')
...
print(compare(list1, list2))
```

Reverse SLL

```
# time O(n), space O(1)
def reverse_sll(head):
    current = head
    previous = None
    next_node = None
    while current:
        next_node = current.next
        current.next = previous
        previous = current
        current = next_node
    return previous
```

Length of linked list

```
def length_LL(LL, head):
    if (not head):
        return 0
    else:
        return 1 +\
    length_LL(LL, head.next)
```

Break cycle

```
def removeLoop(head, loop_node):
    pointer1 = head
    while(1):
        pointer2 = loop_node
        while (pointer2.next != loop_node and\
               pointer2.next != pointer1):
            pointer2 = pointer2.next
        if pointer2.next == pointer1:
            break
        pointer1 = pointer1.next
    pointer2.next = None
```

Reorder List (3 challenges in one)

L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → ...

```
# in-place, time O(n), space O(1)
def reorderList(head: Node) -> None:
    if not head: return
    # FIND MIDDLE
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    # REVERSE SECOND LIST IN-PLACE
    prev, curr = None, slow
    while curr:
        curr.next, prev, curr = prev, curr, curr.next
    # MERGE 1->6->2->5->3->4
    first, second = head, prev
    while second.next:
        first.next, first = second, first.next
        second.next, second = first, second.next
```

21. Merge Two Sorted Lists

```
# t.c. O(n+m), s.c. O(1)
from typing import Optional
def mergeTwoLists(l1: Optional[ListNode],
                  l2: Optional[ListNode],
                 ) -> Optional[ListNode]:
    prehead = ListNode(0)
    current = prehead
    while l1 and l2:
        if l1.val <= l2.val:
            current.next = l1 #entire node
            l1 = l1.next
        else:
            current.next = l2
            l2 = l2.next
        current = current.next
    # l1 or l2 can still have nodes
    current.next = l1 if l1 else l2
    return prehead.next
```

Merge k Sorted Linked Lists

```
# time c. O(Nlogk), k = # lists. Space c. O(1)
def mergeKLists(lists):
    n = len(lists)
    interval = 1
    while interval < n:
        for i in range( 0, n-interval,
                        interval*2 ):
            lists[i] = merge2Lists( lists[i],
                                    lists[i+interval] )
        interval *= 2
    return lists[0] if n > 0 else None

def merge2Lists(l1, l2):
    head = point = Node(0)
    while l1 and l2:
        if l1.val <= l2.val:
            point.next = l1
            l1 = l1.next
        else:
            point.next = l2
            l2 = l1
            l1 = point.next.next
        point = point.next
    if not l1: point.next=l2
    else: point.next=l1
    return head.next
```

Merge k Sorted Arrays

```
# time c. O(kN*Logk) since
# using heap (N*Logk) k times;
# space c. O(N) - output array
from heapq import merge
def mergeK(arr, k):
    l = arr[0]
    for i in range(k-1):
        l = list(merge(l, arr[i + 1]))
    return l
```

2. Add Two Numbers (as linked list)

```python
# t.c. = s.c. = O(max(n, m))
def addTwoNumbers(l1: Node, l2: Node) -> Node:
    prehead = Node(0)                       # A dummy head to simplify handling the result.
    current = prehead                       # Pointer to construct the new list.
    carry = 0                               # carry from each addition
    while l1 or l2 or carry:                # Traverse both lists until exhausted.
        val1 = l1.val if l1 else 0          # If either list ran out of digits, use 0
        val2 = l2.val if l2 else 0
        total = val1 + val2 + carry
        carry = total // 10                 # New carry
        new_digit = total % 10
        current.next = Node(new_digit)
        current = current.next
        if l1: l1 = l1.next                 # Move t
        if l2: l2 = l2.next
    return prehead.next                     # Returr
```

543. Diameter of Binary Tree

Longest path (# edges) betw. any two nodes (thru or not thru root)

```python
# t.c.=O(n), s.c.=O(n)
def diameterOfBinaryTree(root: TreeNode) -> int:
    def longest_path(node):
        if not node: return 0
        nonlocal diameter
        left_path  = longest_path(node.left)
        right_path = longest_path(node.right)
        diameter = max(diameter, left_path+right_path)
        # add 1 for connection to parent
        return max(left_path, right_path) + 1
    diameter = 0
    longest_path(root)
    return diameter
```

138. Copy List with Random Pointer

```python
# t.c.=O(n), s.c.=O(n)
from collections import defaultdict
class Node:
    def __init__(self,x,next=None,random=None):
        self.val = int(x)
        self.next = next
        self.random = random
def copyRandomList(head: Node) -> Node:
        # Nodes as keys, their copies as values
        node_map = defaultdict(lambda: None)
        curr = head #Fill node_map
        while curr:
            node_map[curr] = Node(curr.val)
            curr = curr.next
        # Update pointers of copied nodes
        curr = head
        while curr:
            dup = node_map[curr]
            dup.next = node_map[curr.next]
            dup.random = node_map[curr.random]
            curr = curr.next
        return node_map[head]
```

Backtracking - Words from Phone Number

```
# time = O(N * (4^N)), N = len(digits), space = O(n)
def letterCombinations(digits: str) -> List[str]:
    if len(digits) == 0: return []
    letters = {"2": "abc", "3": "def", "4": "ghi",
               "5": "jkl", "6": "mno", "7": "pqrs",
               "8": "tuv", "9": "wxyz"}
    def backtrack(index, path):
        if len(path) == len(digits):  # base case
            combinations.append("".join(path))
            return
        # letters mapped to current digit
        possible_letters = letters[digits[index]]
        for letter in possible_letters:
            path.append(letter)
            backtrack(index + 1, path)
            path.pop()    # remove before moving on
    combinations = []
    backtrack(0, [])
    return combinations
```

Recursion

t.&s.c. O(n)

```
def fact(n):
    if n < 2:
        return 1
    else:
        return n*fact(n-1)

fact(5)
```

Memoization

M. doesn't help t.c., adds to s.c.

```
def factorial(n):
    if n < 2:
        return 1
    if not n in memo:
        memo[n] = n*factorial(n-1)
    return memo[n]

memo = {}
factorial(1)
```

Fibonnaci dynamic

Unlike for factorial M. helps here

```
# memoization (dynamic)  s.c. space + memo = 2O(n)
# time O(n) - counting sequentially
def fib_dyn(n):
    if n == 0 or n == 1:
        return n
    if not n in memo:
        memo[n] = fib_dyn(n-1) +\
                  fib_dyn(n-2)
    return memo[n]

for n in range(20):
    memo = [None] * (n + 1)
    print(fib_dyn(n), end=', ')
```

Iterative Factorial

```
# t.c. O(n), s.c. O(1)
def factorial(n):
    if n < 0:
        raise ValueError("m")
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result
```

Num ways to cover distance

All ways to cover distance it w/1, 2, 3 steps

```
# time O(n)., space O(n)
def count_dp(dist):
    count = [0] * (dist + 1)
    count[0] = 1
    count[1] = 1
    count[2] = 2
    for i in range(3, dist + 1):
        count[i] = ( count[i-1] +\
                     count[i-2] +\
                     count[i-3] )
    return count[dist];

dist = 20
print( count_dp(dist))
```

121415

Fibonnaci iterative

```
# iterative - tuple unpacking
# time O(n) - counting sequentially
def fib_iter(n):
    a = 0
    b = 1
    for i in range(n):
        a, b = b, a + b
    return a

for i in range(20):
    print(fib_iter(i), end=', ')
```

Fibonnaci recursive

```
# time O(2^n)   because 2 calls for each n. S.c. O(n) –
def fib_rec(n):      depth of recursion stack
    if n == 0 or n == 1:
        return n
    else:
        return fib_rec(n-1) + fib_rec(n-2)

# print 1st 20 Fib. nums
for i in range(20):
    print(fib_rec(i), end=', ')
```

Dijkstra's Algorithm

Shortest path from one vertex to all others

```
# time c. O(V + E*logE), iterative, BFS
import heapq    # min heap of (distance, vertex) pairs
# control iteration order - to pick vertex w/smallest dist
def calculate_distances(graph, start):
    # cost from start to each destination
    distances = {vertex: float('infinity') for vertex in graph}
    distances[start] = 0
    pq = [(0, start)]          # min heap or priority queue
    while len(pq) > 0:
        # pop smallest, maintain heap
        current_distance, current_vertex = heapq.heappop(pq)
        # nodes can be added to pq multiple times; process node
        # first time it's removed from pq
        if current_distance > distances[current_vertex]:
            continue
        for neighbor, weight in graph[current_vertex].items():
            distance = current_distance + weight
            # only if new path is better
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                # push onto heap, maintain heap
                heapq.heappush(pq, (distance, neighbor))
    return distances
```

Min. spanning tree - Prim's algo

S.t. for a graph G = (V,E) is **acyclic subset of E connecting all vertices in V** (sum of edge weights minimized)
Most efficient info flow. There may be several spanning trees - we need to find the minimum one.
Using priority queue to select next vertex for growing graph

```
from collections import defaultdict
import heapq
def min_spanning_tree(graph, start):
    ''' Outputs mst - min. spanning tree '''
    mst     = defaultdict(set)
    visited = set([ start ])
    edges   = [ (cost, start, to)
                for to, cost in graph[start].items() ]
    heapq.heapify(edges)
    while edges:
        cost, frm, to = heapq.heappop(edges)
        if to not in visited:
            visited.add(to)
            mst[frm].add(to)
            for to_next, cost in graph[to].items():
                if to_next not in visited:
                    heapq.heappush(edges, (cost, to, to_next))
    return mst
```

Floyd Warshall Algorithm

Shortest path between all pairs of nodes

```
# time c. O(V^3)
def floyd_warshall(graph):
    dist = list(map(lambda i : list(map(lambda j : j , i)) , graph))
    for k in range(V):
        for i in range(V):          # all vertices as source
            for j in range(V):      # all vertices as destination
                # update dist[i][j] if vertex k on shortest path from i to j
                dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j])
    return dist
```

Coin change (knapsack variant)

```
# recursion + memoization
# arr coins + target amnt => fewest coins as change
def coins_dp(target, coins, known_results):
    min_coins = target
    if target in coins:
        known_results[target] = 1
        return 1
    elif known_results[target] > 0:
        return known_results[target]
    else:
        for i in [c for c in coins if c <= target]:
            num_coins = 1 + coins_dp( target-i,
                                      coins,
                                      known_results,
            if num_coins < min_coins:
                min_coins = num_coins
                known_results[target] = min_coins
    return min_coins
```

```
target = 74
coins = [1,5,10,25]
known_results = [0]*(target+1)
coins_dp(target, coins, known_results)
```

Knapsack DP

```
# max sum of values for subset of weights <= W
# O(N*W) time and space
def knapSack(W, weights, values, n):
    DP = [[0 for x in range(W + 1)] for x in range(n + 1)]
    for i in range(n + 1):
        for w in range(W + 1):
            if i == 0 or w == 0:
                DP[i][w] = 0
            elif weights[i-1] <= w:
                DP[i][w] = max(values[i-1] + DP[i-1]\
                    [w - weights[i-1]],  DP[i-1][w])
            else:
                DP[i][w] = DP[i-1][w]
    return DP[n][W]

values = [60, 100, 120]
weights = [10, 20, 30]
W = 50
print(knapSack(W, weights, values, len(values)))
```

Median of Two Sorted Arrays

```
# time O(log(min(N,M)), space O(1). BIN SEARCH
def median(nums1, nums2):
    # capture edge cases
    if len(nums2) < len(nums1):
        nums1, nums2 = nums2, nums1
    total = len(nums1) + len(nums2)
    half = total // 2
    l, r = 0, len(nums1)-1
    # median is guaranteed
    while True:
        i = (l + r) // 2    # for nums1
        # subtr. 2 - j starts at 0, i starts at 0
        j = half - i - 2    # for nums2
        # overflow of indices
        nums1_left  = nums1[i] if i >= 0 else float("-inf")
        nums1_right = nums1[i+1] if (i+1) < len(nums1) else float("inf")
        nums2_left  = nums2[j] if j >= 0 else float("-inf")
        nums2_right = nums2[j+1] if (j+1) < len(nums2) else float("inf")
        # if correct partition is found
        if nums1_left <= nums2_right and nums2_left <= nums1_right:
            if total % 2:
                return min(nums1_right, nums2_right)
            else:
                return ( max(nums1_left, nums2_left) +\
                         min(nums1_right, nums2_right) ) / 2
        # if no correct partition - arrays are in ascending order
        elif nums1_left > nums2_right:
            r = i - 1
        else:
            l = i + 1
```

Traveling Salesman Problem (TSP)

NP hard problem. There is no known polynomial time solution

```
# Naive approach. Time c. n!
from sys import maxsize
V = 4
def travellingSalesmanProblem(graph, s):
    vertices = []          # all verteces, but source vtx
    for i in range(V):
        if i != s:
            vertices.append(i)
    min_pathweight = maxsize    # min weight Hamiltonian Cycle
    while True:
        current_pathweight = 0 # current Path weight(cost)
        k = s                  # compute current path weight
        for i in range(len(vertices)):
            current_pathweight += graph[k][vertices[i]]
            k = vertices[i]
        current_pathweight += graph[k][s]
        min_pathweight = min(min_pathweight,
                             current_pathweight) #update min
        if not next_permutation(vertices):
            break
    return min_pathweight
```

```
def next_permutation(L):
    n = len(L)
    i = n - 2
    while i >= 0 and L[i] >= L[i + 1]:
        i -= 1
    if i == -1:
        return False
    j = i + 1
    while j < n and L[j] > L[i]:
        j += 1
    j -= 1
    L[i], L[j] = L[j], L[i]
    left = i + 1
    right = n - 1
    while left < right:
        L[left], L[right] = L[right], L[left]
        left += 1
        right -= 1
    return True


# matrix representation of graph
graph = [[0, 10, 15, 20], [10, 0, 35, 25],
         [15, 35, 0, 30], [20, 25, 30, 0]]
start = 0
print(travellingSalesmanProblem(graph, s))
```

Kth Largest Element in Array

```
# time = O(n), space = O(1)
def findKthLargest(nums, k):
    def partition(left, right, pivot_idx):
        pivot = nums[pivot_idx]
        # 1. move pivot to end
        nums[pivot_idx], nums[right] = nums[right], nums[pivot_idx]
        # 2. move all smaller elements to the left
        store_idx = left
        for i in range(left, right):
            if nums[i] < pivot:
                nums[store_idx], nums[i] = nums[i], nums[store_idx]
                store_idx += 1
        # 3. move pivot to its final place
        nums[right], nums[store_idx] = nums[store_idx], nums[right]
        return store_idx

    def select_rec(left, right, k_smallest):
        if left == right:    # base case - 1 elem
            return nums[left]
        pivot_idx = random.randint(left, right)
        # find pivot pos in sorted list
        pivot_idx = partition(left, right, pivot_idx)
        # if pivot in final sorted position
        if k_smallest == pivot_idx:
             return nums[k_smallest]
        elif k_smallest < pivot_idx:    # go left
            return select_rec(left, pivot_idx-1, k_smallest)
        else:                           # go right
            return select_rec(pivot_idx+1, right, k_smallest)
    # kth largest = (n - k)th smallest
    return select_rec(0, len(nums)-1, len(nums)-k)
```

nLog(n) solution – sort, then get kth element

Minimum Window Substring

```
# time/space c. O(n+m)
# min. contig. substr. of s w/all chars from t
from collections import Counter
def minWindow(s, t):
    if not t or not s:
        return ''
    # unique chars in t and curr window
    dict_t = Counter(t)
    curr = {}
    len_t  = len(dict_t)
    len_curr  = 0
    l,r = 0,0
    # length, l, r
    res = float('inf'), None, None
    while r < len(s):
        char = s[r]
        curr[ char ] = curr.get(char, 0) + 1
        if char in dict_t and\
        curr[char] == dict_t[char]:
            len_curr += 1
        while l <= r and len_curr == len_t:
            char = s[l]
            if r - l + 1 < res[0]:
                res = (r - l + 1, l, r)
            curr[ char ] -= 1
            if char in dict_t and\
            curr[char] < dict_t[char]:
                len_curr -= 1
            l += 1
        r += 1
    return '' if res[0]==float('inf') else\
           s[res[1]:res[2]+1]
```

```
tips.assign(tip_rate = tips["tip"] / tips["total_bill"])
tips.loc[tips["tip"] < 2, "tip"] *= 2
```

GROUP BY

```
tips.groupby("sex").size()    # female = 87, male = 90
tips.groupby("day").agg({"tip": "mean", "day": "size"})
```

day	tip	day
Fri	2.586842	19
Sat	2.986897	87

```
# agg() on one column in each df from grouped
df.groupby('Team')['Points'].agg(np.mean)
```

```
Team
Angels      768.000000
Kings       761.666667
```

```
tips.groupby(["smoker", "day"]).agg({"tip": ["size", "mean"]})
```

smoker	day	tip size	tip mean
No	Fri	4	2.562500
	Sat	45	3.052889

```
df.groupby('Team').agg(np.size)
```

Team	Rank	Year	Points
Angels	2	2	2
Kings	3	3	3

```
# MULTIPLE AGGREGATION FUNCTIONS ON ONE COL
df.groupby('Team')['Points'].agg([np.sum, np.mean, np.std])
```

Team	sum	mean	std
Angels	1536	768.000000	134.350288
Kings	2285	761.666667	24.006943

Transformation: df.groupby().transform()

Applied to **group** or **column**, returns an obj **w/same index size**

```
score = lambda x: (x - x.mean()) / x.std()*10
df.groupby('Team').transform(score)
```

	Rank	Year	Points
0	-15.000000	-11.618950	12.843272
1	5.000000	-3.872983	3.020286

Filtration: df.groupby().filter(),

Returns subset of df, filtering data on a criteria

```
df.groupby('Team').filter(lambda x: len(x) >= 3)
```

	Team	Rank	Year	Points
0	Riders	1	2014	876
1	Riders	2	2015	789

```
# MULTIPLE AGGREGATION FUNCTIONS ON MANY COLs
df.groupby('Team')[['Points', 'Rank']].agg([np.sum, np.mean, np.std])
```

	Points			Rank		
Team	sum	mean	std	sum	mean	std
Angels	1536	768.000000	134.350288	5	2.500000	0.707107
Kings	2285	761.666667	24.006943	5	1.666667	1.154701

JOIN

```
DataFrame.merge( right_df,
                 how='inner', # {‘left’, ‘right’, ‘outer’, ‘inner’, ‘cross’}
                 on=None,        # join key(s) - column
                 left_on=None,  # if join key(s) w/different names (but same)
                 right_on=None,
                 left_index=False, # use index as join key(s)
                 right_index=False,
                 sort=False,        # Sort join keys lexicographically
                 suffixes=('_x', '_y'),
                 copy=True,
                 indicator=False,  # col _merge saying source of each row
                 validate=None,    # check merge keys if “1:1”, “1:m”, etc.
               )
```

Remove missing data

```
# Remove rows with ANY missing values
df_dropped_rows = df.dropna()
# Remove columns with ANY missing values
df_dropped_columns = df.dropna(axis=1)
# Remove rows with missing values in specific columns
df_dropped_specific1 = df.dropna(subset=['A'])
# Remove columns with missing values in specific rows
df_dropped_specific2 = df.dropna(subset=[1], axis=1)
```

Fill missing data

```
DataFrame.fillna(
    value=None,  # number/dict/Series/df
    method=None, # 'bfill', 'ffill', None
    limit=None,  # # of ffills/bfills
    axis=None,
    inplace=False,)
```

```
# Fill with a constant value
df_filled_constant = df.fillna(250)
# propagate next valid observation forward
df_filled_ffill = df.fillna(method='ffill')
# propagating next valid observation backward
df_filled_bfill = df.fillna(method='bfill')
# Fill all NaNs in cols w/values
values = {"A": 100, "B": 200, "C": 300,}
print(df.fillna(value=values))
# Fill w/df - replacement at same cols & idx
print(df.fillna(df2))
# Fill w/mean of col
df_filled_mean = df.fillna(df.mean())
# Fill w/median of cols
df_filled_median = df.fillna(df.median())
# Fill with mode of cols (mode() returnes df)
df_filled_mode = df.fillna(df.mode().iloc[0])
# Interpolation (default: linear)
# method: 'index','pad','nearest','zero', etc.
df_interpolated = df.interpolate()
print(df_interpolated)
```

Handling outliers

```
# REMOVE
df_no_outliers = df[(df['A'] >= lower_bound) & (df['A'] <= upper_bound)]
# CAP
df['A'] = np.where(df['A'] > upper_bound, upper_bound, df['A'])
df['A'] = np.where(df['A'] < lower_bound, lower_bound, df['A'])
# IMPUTE
median = df['A'].median()
df['A'] = np.where(df['A'] > upper_bound, median, df['A'])
df['A'] = np.where(df['A'] < lower_bound, median, df['A'])
# SCALE - RobustScaler if robust to outliers
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
df['A'] = scaler.fit_transform(df[['A']])
```

Transforming Data

Z-Score: # standard deviations a data point is from the mean.

z-score = (value - mean) / std_dev => StandardScaler()

```
from scipy.stats import zscore
df['A_log']    = np.log(df['A'])
df['A_zscore'] = zscore(df['A'])
print(df)
```

```
     A  A_capped     A_log  A_zscore
0   10    10.000  2.302585 -0.425312
1  -50   -20.625       NaN -2.115292
```

Identifying outliers using the IQR (Interquartile Range)

```
# Calculate Q1 (25th perc.) & Q3 (75th perc.
Q1 = df['A'].quantile(0.25)
Q3 = df['A'].quantile(0.75)
IQR = Q3 - Q1
# Define outlier boundaries
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR
# Identify outliers
outliers = df[(df['A'] < lower_bound) |\
              (df['A'] > upper_bound) ]
```

Isolation Forest - ML to identify and handle outliers

```
from sklearn.ensemble import IsolationForest
iso = IsolationForest(contamination=0.1)
df['outliers'] = iso.fit_predict(df[['A']])
outliers = df[df['outliers'] == -1]
```

```
     A  A_capped     A_log  A_zscore  A_median  A_scaled  outliers
0   10    10.000  2.302585 -0.425312      10.0 -0.473118         1
1  -50   -20.625       NaN -2.115292      21.0 -3.053763        -1
```

Read CSV files

pandas.read_csv(

- **filepath**, **sep**=',' / regex, **header**='infer' / None / row number (numbers for multi-index),
- **names**=col names. If they already exist in file, pass header=0 to rename,
- **index_col**=col(s) for row labels (None), **usecols**=subset of cols to use (None)
- **dtype**=dtypes for cols (None), **skiprows**=line numbers to skip (0-indexed) or # lines to skip (int)
- **skipfooter**=# lines at bottom of file to skip (0), **nrows**=# rows to read (None),
- **na_values**=additional strings to recognize as NaN, (also **true_values** and **false_values**),
- **skip_blank_lines**=True, **parse_dates**=None, **infer_datetime_format**=_NoDefault.no_default,
- **keep_date_col**= keep original date col, **date_format**=None,
- **thousands**=None, **decimal**='.',**quotechar**='"', **quoting**=0 (csv.QUOTE_MINIMAL, etc.),
- **escapechar**=None, **encoding**=None,
- **encoding_errors**='strict' / 'ignore' / 'replace', **on_bad_lines**='error' / 'warn' / 'skip',
- **delim_whitespace**=whether or not whitespace will be used as the sep (equ. to sep='\s+')

```
df.to_json(file_name)
df = pd.read_json(file_name)
```

Rank

Rank col values in descend or ascend order
Method - How rank group of records
that have the same value (ties):

- average: average rank of group
- min: lowest rank in group
- max: highest rank in group
- first: in order ranks appear
- dense: = 'min', but rank increases by 1

Axending=False => max value has rank 1

```
df['rank'] = df['num_legs_1178'].rank(
    method='first',
    ascending=False).astype("int")
df[['num_legs_1178', 'rank']]
```

	num_legs_1178	rank
falcon	2	3
dog	4	2
spider	8	1
fish	0	4

Set value for a cell

```
df.at['C', 'a'] = 10 # BY VALUE
df.iat[5,10] = 2     # BY INDEX
```

Select rows and cols

Rows & cols = scalars,lists,slice obj,bools
Simult. rows & cols - rows left, cols right of ","

```
# BY POSITION (2ND ROW)
df.iloc[1]
positions = [0, 2, 3]
df.iloc[positions]
# SLICING - EXCLUSIVE (as lists)
df.iloc[1:4, 1:4]

# BY INDEX
df.loc['dog']
idxs=['falcon','spider','fish']
df.loc[idxs]
# SLICING - INCLUSIVE
df.loc['dog':'fish','height':'weight']

# ALL ROWS+COLS 1&4
df.iloc[:,[1,4]]
rows = [0, 2, 3]
cols = [2, 3, 4]
df.iloc[rows, cols]

# SAME W/LOC
df.loc[:,['rating','spec']]
rows = ['falcon', 'dog',]
cols = ['legs', 'wings', 'spec']
df.loc[rows, cols]
```

Select columns

```
df.dtypes.value_counts()
# col by type
df.select_dtypes(include=['int'])
# col by name - has "num" in name
df.filter(like='num')
# has digit(s) in col name
df.filter(regex='\d')
```

	num_legs_1178
falcon	2

Select w/"get_loc" and "index"

```
start = df.columns.get_loc('num_legs_1178')
end   = df.columns.get_loc('num_specimen')
df.iloc[:4, start:end+1]

start = df.index.get_loc('dog')
end   = df.index.get_loc('fish')
df.iloc[start:end+1, 2:5]
```

```
# same things
df[df['num_legs_1178'] > 2]
df.query('num_legs_1178 > 2')
```

Other

```
# cross-tabulation of 2+ factors
pd.crosstab(
    df["genus_overall"],
    df["rating_overall"],
    margins=True,
    normalize=0,)

# SQLite
import sqlite3
query = 'SELECT * FROM dune_table'
conn  = sqlite3.connect('dune.db')
dune_df = pd.read_sql(query, conn)
dune_df.to_sql('dune_table', conn,
               index=False)

# reshape w/new cols & values + aggr.
pivot_df = pd.pivot_table(
    df, index='Name',
    columns=['Pets', 'Stores'],
    values='Weight'  # col for values
    aggfunc='mean')
```

rating_overall	1.2	3.4	5.2	7.8
genus_overall				
avian	1.00	0.00	0.00	0.00
canine	0.00	1.00	0.00	0.00

Melt - unpivot df from wide to long format

```
>>> df
   A  B  C
0  a  1  2
1  b  3  4
2  c  5  6
```

```
>>> pd.melt(df, id_vars=['A'], value_vars=['B', 'C'])
   A variable  value
0  a        B      1
1  b        B      3
2  c        B      5
3  a        C      2
4  b        C      4
5  c        C      6
```

1a. Aggregations on entire dataframe

WINDOW FUNCTIONS - find trends in data graphically by smoothing the curve

- **df.rolling()** - rolling window calculations; **window**=window size, **min_periods**=min num observations in window required to have a value.
- **df.expanding()** - same as rolling, but uses all the data up to that point in time. Equivalent statements: [df.rolling(window=len(df), min_periods=1).mean()] = [df.expanding(min_periods=1).mean()]
- **df.ewm()** - exponentially weighted window similar to expanding window, but each prior point is exponentially weighted down relative to the current point

```
df['column'].rolling(2).sum()
df['column'].rolling(2, min_periods=1).sum()
df['column'].rolling(2).mean()
```

```python
# SIMPLE TEST OF BINARY SEARCH
import unittest
from practice import binary_search

CASES = [
    ([2, 4, 7, 8, 9, 10], 10, 5),
    ([1, 2], 2, 1),
    ([1], 1, 0),
    ([], 1, None),
]

class TestBinarySearch(unittest.TestCase):
    def test_search(self):
        for arr, value, expected in CASES:
            with self.subTest(arr=arr, value=value):
                self.assertEqual(binary_search(arr, value), expected)
```

TO RUN
python -m unittest –v # all
python -m unittest -v test_practice.py
-v = verbose

```python
import unittest
# Class containing unit tests
class TestStringMethods(unittest.TestCase):
    def setUp(self):
        # Setup required resources
        self.string = "hello world"
        self.empty_string = ""
        self.whitespace_string = "   "
        self.numeric_string = "12345"
    def tearDown(self):
        # Clean up resources
        pass
    def test_upper(self):
        self.assertEqual(self.string.upper(),
                         "HELLO WORLD")
    def test_isupper(self):
        self.assertFalse(self.string.isupper())
        self.assertTrue("HELLO".isupper())
    def test_islower(self):
        self.assertTrue(self.string.islower())
        self.assertFalse("Hello".islower())
    def test_split(self):
        self.assertEqual(self.string.split(),
                         ["hello", "world"])
        with self.assertRaises(TypeError):
            s.split(2)
# Entry point for running the tests
if __name__ == '__main__':
    unittest.main()
```

Code testing

Types of tests

- **Unit test** - finding bugs in *individual functions*.
  Should be considered together with integration test -
  when a system is comprehensively unit tested,
  it makes integration testing far easier
- **Integration test** - test *multiple components* of application at once,
  incl. interactions between the parts - identifies defects in the interfaces
  between disparate parts of the codebase.
- **Functional test** - *whole system test*. Sometimes it's aka
  integration test, sometimes - user test

Basics of unittest library

What you need for a unit test:

- **test fixture** - preparations for test: creating temporary or proxy databases,
  directories, or starting a server.
- **test case** - individual unit of testing, checks response to a particular set of inputs
  (base class, TestCase, to create new test cases).
- **test suite** - collection of test cases, test suites, or both: to aggregate tests together.
- **test runner** - orchestrates the execution of tests and provides the outcome.

Additional ¶

Main assertions: *assertEqual(), assertTrue(), assertFalse(), assertRaises()*
unittest.main() - command-line interface to run the test script
Test discovery via CLI finds all unittest files in the directory structure
Unittest can *skip individual tests* and whole classes of tests, marking a test as
an "expected failure," (shouldn't be counted as a failure on a TestResult)
Unittest *can distinguish small differences in tests* (e.g. in some parameters)
using the subTest() context manager. *Doctest* can test docstrings

PYTEST

```python
# SIMPLE TEST OF BINARY SEARCH
import pytest
from practice import binary_search

# use decorator to parametrize test cases
@pytest.mark.parametrize("arr,value,expected", [
    ([2, 4, 7, 8, 9, 10], 10, 5),
    ([1, 2], 2, 1),
    ([1], 1, 0),
    ([], 1, None),
])
def test_binary_search(arr, value, expected):
    assert binary_search(arr, value) == expected
```

```python
import pytest    # lighter-weight syntax vs. unittest

# Fixture for setup and teardown
@pytest.fixture
def string_data():
    data = { "string": "hello world",
             "empty_string": "",
             "whitespace_string": "   ",
             "numeric_string": "12345", }
    return data

# Test functions
def test_upper(string_data):
    assert string_data["string"].upper() == "HELLO WORLD"

def test_isupper(string_data):
    assert not string_data["string"].isupper()
    assert "HELLO".isupper()

def test_islower(string_data):
    assert string_data["string"].islower()
    assert not "Hello".islower()

def test_split(string_data):
    assert string_data["string"].split() == ["hello", "world"]
    assert string_data["string"].split('o') == ["hell", " w", "rld"]

# Run the tests
if __name__ == '__main__':
    pytest.main()
```

TO RUN
python -m pytest –v # all
python -m pytest -v test_practice.py
-v = verbose

Type Hints

```python
from typing import Optional

# var of specified type or None
def get_user_email(user_id: int) -> Optional[str]:
    emails = {1: "alice@example.com", 4: "bob@example.com"}
    return emails.get(user_id, None)

email = get_user_email(1)
print(email)              # Output: alice@example.com

email = get_user_email(2)
print(email)              # Output: None
```

```python
from typing import Any

# any type. E.g. Dict[str, Any]
def print_value(value: Any) -> None:
    print(f"Value: {value}")

print_value(42)          # Output: Value: 42
print_value("Hello")     # Output: Value: Hello
print_value([1, 2, 3])   # Output: Value: [1, 2, 3]
```

```python
from typing import Union

# var is one of several types
def process_data(data: Union[int, str]) -> str:
    if isinstance(data, int):
        return f"Processed integer: {data}"
    elif isinstance(data, str):
        return f"Processed string: {data}"
```

```python
from typing import Callable

# var is func or method
def execute_function(
    func: Callable[[int, int], int],
    a: int,
    b: int,
) -> int:
    return func(a, b)

def add(x: int, y: int) -> int:
    return x + y

result = execute_function(add, 5, 3)
print(result)  # Output: 8
```

```python
import openai
import tiktoken
import backoff

@backoff.on_exception(backoff.expo, openai.error.RateLimitError, max_time=10)
def get_response(model, messages, temperature=0.5, max_tokens=None):
    '''Send request, return reponse'''
    response  = openai.ChatCompletion.create(
        model = model,
        messages = messages,
        temperature = temperature,        # range(0,2), the more the less deterministic / focused
        top_p = 1,                        # top probability mass, e.g. 0.1 = only tokens from top 10% proba mass
        n = 3,                            # number of chat completions
        #max_tokens = max_tokens,          # tokens to return
        stream = False,
        stop=None,                        # sequence to stop generation (new line, end of text, etc.)
        )
    content = [response['choices'][0]['message']['content'].strip(),
               response['choices'][1]['message']['content'].strip(),
               response['choices'][2]['message']['content'].strip(),]
    #num_tokens_api = response['usage']['prompt_tokens']
    return content

def get_gpt_title(text_, prompt_):
    '''Classify text_ using prompt_ and ChatGPT API'''

    # compose messages and reduce length of chapter if needed
    messages = [
            { "role": "system", "content": "You are a helpful title generator.", },
            { "role": "user", "content": prompt_.format(text_), },
            ]
    while not verify_num_tokens(model, messages):
        text_len = len(text_)
        text_ = text_[:text_len//2]
        messages = [
            { "role": "system", "content": "You are a helpful title generator.", },
            { "role": "user", "content": prompt_.format(text_), },
            ]

    return get_response(model, messages, )

prompt_one = """
1. Please generate a concise title for the text delimited below with triple backticks
2. The maximum length of title should be 5 words

Text:
```{}```
"""
s = 'This is a text sample'
print(prompt_one.format(s), '\n')

#openai.api_key = ""
#openai.api_key = os.getenv("OPENAI_API_KEY")
model          = 'gpt-3.5-turbo'

text_col = 'text_merged'
#res      = dict()
count    = 0
for text_ in df[text_col].tolist():
    if t in res:
        continue
    try:
        res[ text_ ] = get_gpt_title(text_, prompt_one)
    except openai.error.RateLimitError:
        print(f'\nText: {text_}.\nRate limit error\n')
    except Exception as e:
        print(f'\nText: {text_}\nError: {e}\n')
    count += 1
    if count % 10 == 0:
        print(f'Processing chapter #{count}')
```
```

Summarize Transcript

```python
from transformers import T5Tokenizer, T5ForConditionalGeneration
import torch

# The GPU 15GB can hold the whole model but not the CPU 13GB
torch.set_default_tensor_type(torch.cuda.FloatTensor)

tokenizer = T5Tokenizer.from_pretrained("google/flan-t5-xl")
model_t5 = T5ForConditionalGeneration.from_pretrained("google/flan-t5-xl", device_map="auto")

def generate(transcript, instruction, n=3):
    '''Original prompt that came with this notebook'''
    transcript = '\n'.join(transcript)[:1500]
    input_text = f'## Transcript\n\n"{transcript}"\n\n## Directions\n\n{instruction}'
    input_ids = tokenizer(input_text, return_tensors="pt").input_ids
    outputs = model_t5.generate(input_ids, max_new_tokens=100, num_return_sequences=n, num_beams=n)
    return [tokenizer.decode(o).replace('</s>', '').replace('<pad>', '').strip() for o in outputs]

segment_transcript = [ "Alice: Hello, this meeting is about notes.",
                       "Bill: Yes, the notes taken during a meeting",
                       "Alice: And we must summarize these notes",
                       "Bill: Good morning, yes.",
                       "Alice: Okay that's everything."]
instruction = "In 1 to 5 words, generate a title for the above transcript"
print(generate(segment_transcript, instruction))
```

Reflexion Agent 1 (LangGraph)


```python
import asyncio
from langgraph.graph import END, MessageGraph
from typing import List, Sequence
from langgraph.graph import MessageGraph, END
from langchain_core.messages import BaseMessage, HumanMessage
from decouple import config
from langchain_openai import ChatOpenAI
from typing import List
from langchain_core.messages import AIMessage, BaseMessage, HumanMessage
from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages(
    [( "system",
       "You are an AI assistant researcher tasked with researching on a variety of topics in a short summary of 5 pa
       " Generate the best research possible as per user request."
       " If the user provides critique, respond with a revised version of your previous attempts.", ),
        MessagesPlaceholder(variable_name="messages"), ])
llm = ChatOpenAI(api_key=config("OPENAI_API_KEY"))
generate = prompt | llm

reflection_prompt = ChatPromptTemplate.from_messages(
    [( "system",
       "You are a senior researcher"
       "Provide detailed recommendations to an asistant researcher to help improve this researches",),
        MessagesPlaceholder(variable_name="messages"),])
reflect = reflection_prompt | llm

async def generation_node(state: Sequence[BaseMessage]):
    return await generate.ainvoke({"messages": state})

async def reflection_node(messages: Sequence[BaseMessage]) -> List[BaseMessage]:
    # Other messages we need to adjust
    cls_map = {"ai": HumanMessage, "human": AIMessage}
    # First message is the original user request. We hold it the same for all nodes
    translated = [messages[0]] + [ cls_map[msg.type](content=msg.content) for msg in messages[1:] ]
    res = await reflect.ainvoke({"messages": translated})
    # this will be treated as a feedback for the generator
    return HumanMessage(content=res.content)
```

Reflexion Agent 2 (LangGraph)


```python
builder = MessageGraph()
builder.add_node("generate", generation_node)
builder.add_node("reflect", reflection_node)
builder.set_entry_point("generate")

def should_continue(state: List[BaseMessage]):
    if len(state) > 6: return END    # End after 5 iterations
    return "reflect"

builder.add_conditional_edges("generate", should_continue)
builder.add_edge("reflect", "generate")
graph = builder.compile()

async def stream_responses():
    async for event in graph.astream([
            HumanMessage( content="Research on climate change" )], ):
        print(event, "\n---")

asyncio.run(stream_responses())
```

Iterative self-refinement system - **Self-critique loop** - AI iteratively improves its output by *alternating between generation and reflection roles*, simulating *peer review*. AI researcher **Generator Agent** generates content and receives feedback / critique senior researcher **Reflector Agent** to improve.

Prompt Templates

- **Generator prompt**: Instructs AI to create 5-paragraph research and incorporate feedback
- **Reflection prompt**: Instructs AI to provide improvement recommendations

Graph-Based Workflow (LangGraph)

- **Cyclic graph**: generate → reflect → generate → ...
- **Conditional edge** stops after 6+ messages (~3 iterations)

Message Role Swapping (Key Technique)

- In reflection_node: Flips message roles (AI ↔ Human)
- **Reflector** *perceive previous AI outputs as human inputs* to critique AND
- *Returns critique as HumanMessage* so **generator** treats it as user feedback

Async Streaming of events

Multi-Agent Collaboration Agent 1 (LangGraph)

```python
import asyncio
from langgraph.graph import StateGraph, END
from typing import TypedDict, Annotated
from langchain_core.messages import HumanMessage, SystemMessage
from langchain_openai import ChatOpenAI
from decouple import config

# Shared state that all agents will use
class AgentState(TypedDict):
    task: str
    research_notes: str
    draft_content: str
    final_output: str
llm = ChatOpenAI(api_key=config("OPENAI_API_KEY"), temperature=0.7)

async def researcher_agent(state: AgentState):
    prompt = f"""You are a research agent. Your task is to gather key facts,
    statistics, and important points about the following topic:
    Topic: {state['task']}
    Provide 5-7 bullet points with relevant information."""

    response = await llm.ainvoke([HumanMessage(content=prompt)])
    return { **state, "research_notes": response.content }

async def writer_agent(state: AgentState):
    prompt = f"""You are a content writer. Using the research notes below,
    write a well-structured blog post (3-4 paragraphs) about:
    Topic: {state['task']}
    Research Notes:
    {state['research_notes']}
    Write engaging, clear content."""

    response = await llm.ainvoke([HumanMessage(content=prompt)])
    return { **state, "draft_content": response.content }
```

This pattern demonstrates **task decomposition** where a complex task is **split into subtasks handled by specialized agents**

Shared State: AgentState dictionary passed between all agents

Three Specialized Agents:
Researcher: Gathers information and facts
Writer: Creates draft content from research
Editor: Polishes and finalizes the output
Sequential Collaboration: Each agent builds upon the previous agent's work

StateGraph Pattern: Linear workflow with clear handoffs between agents

Alternative: Parallel + Aggregator Pattern

If you want agents working in parallel instead:

```python
# Combines outputs from parallel agents
async def aggregator_agent(state: AgentState):
    pass

# Modify workflow
workflow.add_edge("researcher", "aggregator")
workflow.add_edge("writer", "aggregator")
workflow.add_edge("editor", "aggregator")
```

```python
async def editor_agent(state: AgentState):
    prompt = f"""You are an editor. Review and improve the following draft.
    Fix any issues, enhance clarity, and ensure professional quality.
    Original Topic: {state['task']}
    Draft:
    {state['draft_content']}
    Provide the polished final version."""

    response = await llm.ainvoke([HumanMessage(content=prompt)])
    return {**state, "final_output": response.content }

def create_workflow():
    builder = StateGraph(AgentState)
    # Add the three agent nodes
    builder.add_node("researcher", researcher_agent)
    builder.add_node("writer", writer_agent)
    builder.add_node("editor", editor_agent)
    # Define the sequential flow: researcher → writer → editor → END
    builder.set_entry_point("researcher")
    builder.add_edge("researcher", "writer")
    builder.add_edge("writer", "editor")
    builder.add_edge("editor", END)
    return builder.compile()

# Execute the multi-agent collaboration
async def main():
    graph = create_workflow()
    # Initial task
    initial_state = { "task": "The benefits of meditation for mental health",
                      "research_notes": "", "draft_content": "", "final_output": "", }
    # Run the workflow
    result = await graph.ainvoke(initial_state)
    # Display results
    print("\n📊 RESEARCH NOTES:")
    print(result["research_notes"])
    print("\n📝 DRAFT CONTENT:")
    print(result["draft_content"])
    print("\n✅ FINAL OUTPUT:")
    print(result["final_output"])

if __name__ == "__main__":
    asyncio.run(main())
```

Multi-Agent Collaboration Agent 2 (LangGraph)

OpenAI Agents Framework: Self-Reflection

```python
# OpenAI Agents framework
import json
from agents import Agent, Runner
from pydantic import BaseModel, Field

class Critique(BaseModel):
    valid: bool = Field(description="True if orig. answer already correct, complete, and clear")
    rewrite: str = Field(description="An improved answer if valid is False. Empty string otherwise.")

# 2. Define the two agents
answer_agent = Agent(
    name="AnswerAgent",
    instructions=('You are a knowledgeable assistant, think step-by-step',
                  'return final concise answer', ), )

reflection_agent = Agent(
    name="ReflectionAgent",
    instructions=("You are a strict reviewer. Evaluate the answer ..."),
    output_type=Critique, )

def self_reflect(question: str, max_iters: int = 3) -> str:
    answer_result = Runner.run_sync(answer_agent, question)
    for _ in range(max_iters):
        critique_prompt = (f"QUESTION:\n{question}\n\nANSWER:\n{answer_result.final_output}")
        critique_result = Runner.run_sync(reflection_agent, critique_prompt)
        critique = critique_result.final_output_as(Critique)
        if critique.valid:
            return answer_result.final_output
        improvement_prompt = (
            f"The reviewer says the answer needs improvements:\n{critique.rewrite}\n\n"
            f"Write a **better** answer that resolves all issues." )
        answer_result = Runner.run_sync(answer_agent, improvement_prompt)
    return answer_result.final_output    # fallback – the best

user_question = "Explain why the sky appears blue during the day?"
final_answer = self_reflect(user_question)
print(final_answer)
```

LangChain Framework: Self-Reflection


```python
from langchain.chat_models import ChatOpenAI
from langchain.prompts import PromptTemplate
from langchain.chains import LLMChain
from langchain.output_parsers import PydanticOutputParser
from pydantic import BaseModel, Field

# Schema for reviewer output
class Critique(BaseModel):
    valid: bool = Field(description="True if orig. answer already correct, complete, and clear")
    rewrite: str = Field(description="An improved answer if valid is False. Empty string otherwise.")

critique_parser = PydanticOutputParser(pydantic_object=Critique)

# Build the two chains
llm = ChatOpenAI(model="gpt-4o-mini", temperature=0.3)
answer_prompt = PromptTemplate.from_template( 'You are ...' )
answer_chain = LLMChain(llm=llm, prompt=answer_prompt)

reflection_prompt = PromptTemplate(
    template=(
        "You are a strict reviewer. Evaluate the answer ...'
        f"Respond ONLY with JSON matching this schema:\n{critique_parser.get_format_instructions()}"),
    input_variables=["question", "answer"],
    output_parser=critique_parser, )
reflection_chain = LLMChain(llm=llm, prompt=reflection_prompt, output_parser=critique_parser)

def self_reflect(question: str, max_iters: int = 3) -> str:
    answer = answer_chain.run(question=question).strip()
    for _ in range(max_iters):
        critique = reflection_chain.run(question=question, answer=answer)
        if critique.valid:
            return answer
        answer = answer_chain.run(
            question=f"The reviewer says to improve the answer like this:\n{critique.rewrite}"
        ).strip()
    return answer

q = "Explain why the sky appears blue during the day?"
final = self_reflect(q)
print("\nFinal answer after reflection:\n", final)
```

Vertex AI API

```
# ONE TEXT API CALL
import vertexai
from vertexai.generative_models import GenerativeModel, Part, Image

# Initialize Vertex AI
PROJECT_ID = "your-gcp-project-id"
LOCATION = "us-central1"
vertexai.init(project=PROJECT_ID, location=LOCATION)

# Load the Gemini 1.0 Pro model
model = GenerativeModel(model_name="gemini-1.0-pro-002")

# Send a prompt to the model
prompt = "Define a VPN and a proxy in simple terms."
response = model.generate_content(prompt)

print(response.text)
```

```
# MULTIMODAL CALL
vision_model = GenerativeModel(model_name="gemini-1.0-pro-vision-001")

# Create an image Part from a Google Cloud Storage URI
image = Part.from_uri(    mime_type="image/jpeg",
                          uri="gs://cloud-samples-data/landmark.jpg",    )
# Combine image + text
prompt_parts = [    image,
                    "What famous landmark is this?",    ]

response = vision_model.generate_content(prompt_parts)
print(response.text)
```

Google AI Studio API

```
import google.generativeai as genai

# Configure API
GOOGLE_API_KEY = "YOUR_GOOGLE_AI_STUDIO_API_KEY"
genai.configure(api_key=GOOGLE_API_KEY)

# Select the model
model = genai.GenerativeModel(model_name="gemini-1.0-pro")

prompt = "Define a Python learning course"
response = model.generate_content(prompt)
print(response.text)

# Generate content with STREAMING ENABLED
prompt = "Write a futuristic story about robots."
response_stream = model.generate_content(prompt, stream=True)

for chunk in response_stream:
    print(chunk.text, end="")
print("\n--- Story End ---")
```

FINE-TUNING ON 4 GPUs (HF & PyTorch)

- **Multiple NVIDIA GPUs** (4x A100, V100, or RTX 4090s)
- **Connected via NVLink** (NVidia proprietory GPU-GPU interconnect) or **PCIe (standard GPU-CPU bus)** (interconnect technologies that allow GPUs to communicate)
- Use **Distributed Data Parallel (DDP)** if model fits **on 1 GPU** - replicate across all GPUs:
  - Each GPU gets *a copy of model*, *data split* across GPUs as batches
  - *Gradients are synchronized* after backward pass
  - Handled automatically by TrainingArguments when multiple GPUs detected
- Use **FSDP (Fully Sharded Data Parallel)** if model **too large for 1 GPU** - shard across all GPUs
- **PyTorch handles synchronization**, you just launch with torchrun

To tun the below code:
`torchrun --nproc_per_node=4 train.py` # For 4 GPUs on one machine

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer, TrainingArguments, Trainer
from datasets import load_dataset
from torch.nn.parallel import DistributedDataParallel as DDP

# Load model and tokenizer
model_name = "meta-llama/Llama-2-7b-hf"
model = AutoModelForCausalLM.from_pretrained(model_name)
tokenizer = AutoTokenizer.from_pretrained(model_name)

# Prepare dataset
dataset = load_dataset("your_dataset")
def tokenize_function(examples):
    return tokenizer(examples["text"], truncation=True, max_length=512)
tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Training arguments for multi-GPU
training_args = TrainingArguments(
    output_dir="./output",
    per_device_train_batch_size=4,
    gradient_accumulation_steps=4,
    num_train_epochs=3,
    learning_rate=2e-5,
    fp16=True,  # Mixed precision training
    ddp_find_unused_parameters=False,
    # Multi-GPU settings are auto-detected )

# Train
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],)
trainer.train()

# For larger models (FSDP - Fully Sharded Data Parallel):
training_args = TrainingArguments(
       fsdp="full_shard auto_wrap",  # Shard model across GPUs
       fsdp_config={
           "fsdp_transformer_layer_cls_to_wrap": ["LlamaDecoderLayer"]})
```

INFERENCE ON 4 GPUs

- Simpler (different strategies) - **no distributed training / no gradients**! Just *forward passes*
- Load model w/device_map="auto" and you're done

```python
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-13b-hf",
    device_map="auto",           # HANDLES MULTI-GPU AUTOMATICALLY
    torch_dtype=torch.float16,
    low_cpu_mem_usage=True
)
tokenizer = AutoTokenizer.from_pretrained("meta-llama/Llama-2-13b-hf")

# Just run it - framework handles GPU distribution
inputs = tokenizer("Hello, how are you?", return_tensors="pt").to("cuda")
outputs = model.generate(**inputs, max_length=50)
print(tokenizer.decode(outputs[0]))
```

Common Inference Strategies

1. **Data Parallelism** (*Model fits on 1 GPU*)
   Assign **model copy to each GPU** manually, distribute requests across GPUs

```
...
device = torch.device(f"cuda:{gpu_id}")  # gpu_id = 0, 1, 2, 3...
model.to(device)
...

In production use vLLM, TGI

# vLLM automatically handles multi-GPU data parallelism
from vllm import LLM
llm = LLM(
    model="meta-llama/Llama-2-7b-hf",
    tensor_parallel_size=4  # Use 4 GPUs, )
outputs = llm.generate(["prompt1", "prompt2", ...])
```

2. **Tensor Parallelism** (*Model too large for 1 GPU*)
   Split model layers across GPUs

```
from transformers import AutoModelForCausalLM
# Automatically shard model across all GPUs
model = AutoModelForCausalLM.from_pretrained(
    "meta-llama/Llama-2-70b-hf",
    device_map="auto",  # Automatically distributes layers
    torch_dtype=torch.float16,)
```

3. **Pipeline Parallelism**
   Different stages of model on different GPUs, process multiple requests in pipeline:

```
from transformers import pipeline
# Pipeline automatically handles device placement
pipe = pipeline(
    "text-generation",
    model="model_name",
    device_map="auto",)
outputs = pipe("Your prompt")
```

DEEPSPEED

Microsoft's optimization lib to makes multi-GPU training/inference faster and more memory-efficient than standard PyTorch.

- Create deepspeed config (batch size, gradient_accumulation_steps, fp16 & other params)
- In PyTorch, add it to TrainingArguments() and import deepspeed
- Run it with `deepspeed --num_gpus=4 train.py`

TRAINING

✅ Use DeepSpeed when:

- Model doesn't fit in GPU memory (even with FSDP)
- Training very large models (70B+ parameters)
- Need CPU/NVMe offloading
- Want maximum memory efficiency

❌ Stick with PyTorch FSDP when:

- Smaller models (<13B parameters)
- Want simpler setup Using models without DeepSpeed kernel support

INFERENCE

✅ Use DeepSpeed when:

- Need maximum throughput
- Latency-critical applications
- Large models requiring tensor parallelism

❌ Use alternatives (vLLM, TGI) when:

- Want easier setup with better defaults
- Need dynamic batching
- Production serving (vLLM is more popular)

```python
import argparse

parser = argparse.ArgumentParser(description="Greet someone.")
parser.add_argument("name", help="who to greet")
parser.add_argument("-n", "--count", type=int, default=1, help="repetitions")
parser.add_argument("-v", "--verbose", action="store_true")

args = parser.parse_args()

if args.verbose:
    print(f"Greeting {args.name} {args.count}x")
for _ in range(args.count):
    print(f"Hello, {args.name}!")

# Key bits: positional args have no dash, optional ones do;
# action="store_true" makes a flag;
# type= coerces and validates.
```

```python
# TUPLE
t = (1, 2, 2)
(1,)        # single element — comma required
t.count(2)
t.index(2)
x, *rest = t   # unpacking

# immutable & hashable → dict keys, set members
# but: t=([1], 2) — unhashable (mutable element)
```

```python
# SET
s = {1, 2, 3}
# immutable, hashable → used in sets / dict keys
frozenset({1, 2})    s.add(4)
s.remove(9)            # KeyError if absent
s.discard(9)           # no error
s.pop()                # arbitrary element
s | t   s.union(t)
s & t   s.intersection(t)
s - t   s.difference(t)
s ^ t   s.symmetric_difference(t)
s <= t  s.issubset(t)
s.isdisjoint(t)    # True if s & t share no elems
# operators (|, & etc.) need both to be sets sets;
# methods accept any iterable: s.union([1,2]) OK

s = {1, 2}; t = {2, 3}
s |= t # s = {1, 2, 3} — union ≡ s.update(t)
# keep only shared ≡ s.intersection_update(t)
s &= t
# remove t's elements ≡ s.difference_update(t)
s -= t
# keep elem in exactly one ≡ s.symmetric_diff_update(t)
s ^= t
```

```python
# LIST
a = [3, 1, 2]
a.append(4)        # O(1) amortized
a.extend([5, 6])   # in place; a += [5,6] same
a.insert(0, 9)     # O(n)
a.pop()            # O(1), from end
a.pop(0)           # O(n) — use deque for queues
# O(n), removes 1st match, ValueError if absent
a.remove(1)
a.index(2)         # O(n), ValueError if absent
a.count(2)
a.reverse()        # in place
a[::-1]            # reversed copy
a.sort()           # in place, returns None!
b = sorted(a)      # new list
a.clear()
a[1:3] = [7, 8]    # slice assignment
```

```python
# DICT
d = {'a': 1}
d['b'] = 2
d.get('x')     # return None if absent
d.get('x', 0) # return default
d['x']         # KeyError if absent
# insert default if missing, return value
d.setdefault('c', []).append(1) # in place
d.pop('a')       # KeyError
d.pop('a', None)# return None — no error
d.popitem()      # removes last inserted (LIFO)
d.update(other) # in place, returns None
d |= other       # same, equivalent
merged = d | other # new dict, d unchanged
d.keys(); d.values(); d.items() # live views, not copies
list(d)          # keys; order = insertion order (3.7+)

from collections import defaultdict, Counter
# no KeyError, auto-creates
defaultdict(list)['k'].append(1)
Counter('aabbc').most_common(2)
# [('a',2),('b',2)]
```

```
# STRING
''.join(parts)   # O(n)
s.split(), s.strip(), s.lower()
s.startswith(t), s.endswith(t)$
# char↔int for freq arrays
ord('a'), chr(97)
s.isdigit(), s.isalpha(), s.isalnum()
```

```
# SORT
a.sort()  # in place
sorted(x) # any iterable, returns list
a.sort(key=len)
# multi-key; negation - descending order
a.sort(key=lambda x: (x[1], -x[0]))
a.sort(reverse=True)
sorted([1, 'a']) # TypeError - mixed types

# strings sort lexicographically: '10' < '9'
# use key=int for numeric strings
# case: sorted(words, key=str.lower)
min(a, key=...), max(a, key=...)
import heapq
# top-k without full sort
heapq.nlargest(3, a, key=...)
```

```
# NAMED TUPLE
from collections import namedtuple
Point = namedtuple('Point', 'x y')
p = Point(1, 2); p.x;
q = p._replace(x=5) # Point(x=5, y=2)

# DATACLASS
from dataclasses import dataclass, field
@dataclass
class Point:
    x: float
    y: float = 0.0
    # defaults follow non-defaults

p = Point(1, 2)
# Point(x=1, y=2) => free __repr_
p
# True => free __eq__ (field-wise)
p == Point(1, 2)
p.x = 5       # mutable by default
```